

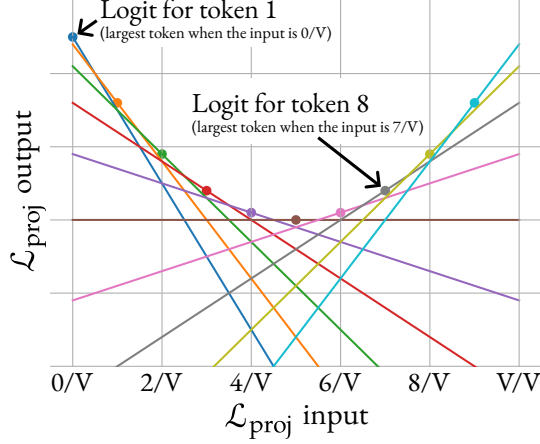


Figure 4: Illustration of the predictors for each token in the $\mathcal{L}_{\text{proj}}$ linear layer for $V = 10$. The layer is constructed in such a way that the i -th token has the highest confidence when the input is $i-1/V$.

construction, the choice of embedding matrix $E \in \mathbb{R}^{N \times V}$ does not matter. The same transformer architecture can generate only V unique outputs if we only control the first token instead. Therefore, it is indeed the case that the embedding space has exponentially more capacity for control than the token space. You can see this transformer implemented and running in practice in Section 2 of [this notebook](#).

A.2 CONDITIONAL GENERATION FOR A SINGLE VIRTUAL TOKEN ($n_X = n_Y = 1$)

This section provides an explicit construction of a transformer with the properties described in Theorem 2. The goal is to construct a transformer that, by varying the choice of the virtual token, can cause the model to act as any map $m : [1, \dots, V] \rightarrow [1, \dots, V]$. In other words, by selecting the virtual token, we can fully control how the model will respond to any token the user may provide.

First, we need to specify how the map m will be encoded in the virtual token s_1 . We choose the embedding size d_e to be V . Now, we can use the same encoding scheme as before, but now each element in s_1 corresponds to a different user token, rather than to a position in the generated sequence:

$$s_1 = (m(1)/V, \dots, m(V)/V).$$

Therefore, the first element of s_1 designates the response if the user provides token 1, the second element is the response to the token 2, and so on.

Extracting the Y_i -th value from s_1 and decoding it can be done in a very similar way as for the unconditional case. The only difference is that instead of looking at the user input position, we look at its value. Take $E = I_V$ and $N = 2$.

Hence we have the following `val` head (only differing in the W_V matrix from Equation (9)):

$$W_Q^{\text{val}} = [0_{2 \times V}, I_2], \quad W_K^{\text{val}} = [0_{2 \times V}, I_2], \quad W_V^{\text{val}} = [I_V, 0_{V \times 2}].$$

We also need embedding of the first token, so we have a modified version of Equation (8):

$$W_Q^{\text{fst}} = [0_V, 1, 1], \quad W_K^{\text{fst}} = [0_V, 1, 0], \quad W_V^{\text{fst}} = [I_V, 0_{V \times 2}].$$

And hence the output of this attention block at the second position would be:

$$t_2 = \sum_{j=1}^2 A_{ij}^{\text{fst}} x_j + \sum_{j=1}^2 A_{ij}^{\text{val}} W_V^{\text{fst}} x_j = s_1 + e_V(Y_1).$$

Similarly to the unconditional case, only the entry of t_2 corresponding to the user token will have a value above 1 and that value would be $1 + m(x_1)/V$.

We can now extract the one-hot representation of the target token using the same approach as before, just adjusting for the different hidden state size: $\hat{\mathcal{L}}_{\text{ex}} = \hat{\mathcal{L}}[\mathbf{I}_V, -\mathbf{1}_V]$, $\hat{\mathcal{L}}_{\text{sum}} = \hat{\mathcal{L}}[\mathbf{1}_V^\top, 0]$, and the same projection had as before (Equation (10)). The final transformer is then: $\mathcal{A}[(\mathbf{W}_Q^{\text{fst}}, \mathbf{W}_Q^{\text{val}}), (\mathbf{W}_K^{\text{fst}}, \mathbf{W}_K^{\text{val}}), (\mathbf{W}_V^{\text{fst}}, \mathbf{W}_V^{\text{val}})] \circ \hat{\mathcal{L}}_{\text{ex}} \circ \hat{\mathcal{L}}_{\text{sum}} \circ \mathcal{L}_{\text{proj}} \circ \text{softmax}$. You can see this transformer implemented and running in practice in Section 3 [here](#).

A.3 CONDITIONAL GENERATION FOR LONGER RESPONSES ($n_X = 1, n_Y > 1$)

We can obtain longer responses via a simple extension. If the response length is N_0 , then we can encode the map $m : [1, \dots, V] \rightarrow [1, \dots, V]^{N_0}$ in N_0 virtual tokens, each corresponding to one of the target positions:

$$\mathbf{s}_i = (m(1)_i/V, \dots, m(V)_i/V) \text{ for } i = 1, \dots, N_0.$$

For this model we would then have $N = 2N_0$ and $d_e = V$.

First, we need a head that always looks at the token provided by the user, which will be at position $N_0 + 1$:

$$\mathbf{W}_Q^{\text{user}} = [\mathbf{0}_V, \mathbf{1}_N], \quad \mathbf{W}_K^{\text{user}} = [\mathbf{0}_{(V+N_0)}, 1, \mathbf{0}_{(N_0-1)}], \quad \mathbf{W}_V^{\text{user}} = [\mathbf{I}_V, \mathbf{0}_{V \times N}].$$

In order to consume the map at the right location, we need to also look at the embedding of the token N_0 positions before the one we are trying to generate:

$$\mathbf{W}_Q^{\text{back}} = \begin{bmatrix} \mathbf{0}_{N_0 \times (N_0+V)} & \mathbf{I}_{N_0} \\ \mathbf{0}_{N_0 \times (N_0+V)} & \mathbf{0}_{N_0 \times N_0} \end{bmatrix}, \quad \mathbf{W}_K^{\text{back}} = [\mathbf{0}_{N \times V}, \mathbf{I}_N], \quad \mathbf{W}_V^{\text{back}} = [\mathbf{I}_V, \mathbf{0}_{V \times N}].$$

From here on, the decoding is exactly the same as in the $n_X = n_Y = 1$ case. The final transformer is then: $\mathcal{A}[(\mathbf{W}_Q^{\text{user}}, \mathbf{W}_Q^{\text{back}}), (\mathbf{W}_K^{\text{user}}, \mathbf{W}_K^{\text{back}}), (\mathbf{W}_V^{\text{user}}, \mathbf{W}_V^{\text{back}})] \circ \hat{\mathcal{L}}_{\text{ex}} \circ \hat{\mathcal{L}}_{\text{sum}} \circ \mathcal{L}_{\text{proj}} \circ \text{softmax}$. You can see this transformer implemented and running in practice in Section 4 [here](#).

A.4 CONDITIONAL GENERATION FOR LONGER USER INPUTS ($n_X > 1, n_Y = 1$)

Finally, we consider the case when the user input $\mathbf{x}$ is longer. This is a bit more complicated because we need to search through a domain of size V^V . We will only consider the case with $n_X = 2$ where we would need two attention layers. A similar approach can be used to construct deeper models for $n_X > 2$. Finally, combining the strategy in the previous section for longer responses with the strategy in this section for longer user inputs allows us to construct transformers that map from arbitrary length user strings to arbitrary length responses.

In order to encode a map $m : [1, \dots, V]^2 \rightarrow [1, \dots, V]$ into a single virtual token we would need a more involved construction than before. Similarly to how we discretized each element of the virtual token $\mathbf{s}_1$ in V levels before, we are going to now discretize it into V^V levels. Each one of these levels would be one of the V^V possible maps from the *second* user token to the response. The first user token would be used to select the corresponding element of $\mathbf{s}_1$. Then this scalar will be “unpacked” into a new vector of V elements using the first attention block. Then, the second user token will select an element from this unpacked vector, which will correspond to the target token.

We construct the virtual token as follows:

$$\mathbf{s}_1 = \left[\sum_{i=1}^V m_1(i) \times \frac{V^{i-1}}{V^V}, \dots, \sum_{i=1}^V m_V(i) \times \frac{V^{i-1}}{V^V} \right],$$

where $m_f(x) = m(f, x)$ is a map from the second user token to the response when the first token is fixed to be f .

An additional change from the previous constructions is that we are going to divide the residual stream into two sections. This is in line with the theory that different parts of the residual stream specialize for different communications needs by different attention heads (Elhage et al., 2021). We will use the first half of the residual stream to extract and “unpack” the correct mapping from second token to target token, while the second half of the residual stream will be used to copy the second token value so that the second attention layer can use it to extract the target. As usual, the embedding

matrix will be the identity matrix: $\mathbf{E} = \mathbf{I}_V$. Finally, for convenience, we will also use a dummy zero virtual token that we will attend to when we want to not attend to anything. This results in context size $N = 4$ with the input being

$$\left(\begin{bmatrix} 0_V \\ \mathbf{e}_N(1) \end{bmatrix}, \begin{bmatrix} \mathbf{s}_1 \\ \mathbf{e}_N(2) \end{bmatrix}, \begin{bmatrix} \mathbf{E}_{:,X_1} \\ \mathbf{e}_N(3) \end{bmatrix}, \begin{bmatrix} \mathbf{E}_{:,X_2} \\ \mathbf{e}_N(4) \end{bmatrix} \right) = \left(\begin{bmatrix} \mathbf{0}_V \\ \mathbf{e}_N(1) \end{bmatrix}, \begin{bmatrix} \mathbf{s}_1 \\ \mathbf{e}_N(2) \end{bmatrix}, \begin{bmatrix} \mathbf{e}_V(X_1) \\ \mathbf{e}_N(3) \end{bmatrix}, \begin{bmatrix} \mathbf{e}_V(X_2) \\ \mathbf{e}_N(4) \end{bmatrix} \right).$$

We want the output at the last position to be the target $m(X_1, X_2)$, that is:

$$\arg \max_{u \in 1, \dots, V} \mathbf{y}_{4,u} = m(X_1, X_2) \text{ for any } m, X_1, X_2.$$

The first attention block will have three attention heads.

As before, we want to extract the value of $\mathbf{s}_1$ that corresponds to the first token the user provided (X_1) and place it in the first half of the residual stream. We want only the third position to do that, while the rest of the positions keep the first half of their residual stream with zeros. Hence we have the following `fst` head:

$$\mathbf{W}_Q^{\text{fst}} = \begin{bmatrix} 0_{2 \times V} & \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{bmatrix}, \quad \mathbf{W}_K^{\text{fst}} = \begin{bmatrix} 0_{2 \times V} & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \end{bmatrix}, \quad \mathbf{W}_V^{\text{fst}} = \begin{bmatrix} \mathbf{I}_V & 0_{V \times N} \\ 0_{V \times V} & 0_{V \times N} \end{bmatrix}.$$

The `user1` head extracts the value of the first user-provided token (X_1) and also places it in the first half of the residual stream:

$$\mathbf{W}_Q^{\text{user1}} = \begin{bmatrix} 0_{2 \times V} & \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{bmatrix}, \quad \mathbf{W}_K^{\text{user1}} = \begin{bmatrix} 0_{2 \times V} & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \end{bmatrix}, \quad \mathbf{W}_V^{\text{user1}} = \begin{bmatrix} \mathbf{I}_V & 0_{V \times N} \\ 0_{V \times V} & 0_{V \times N} \end{bmatrix}.$$

And the `user2` head does the same for the value of the second user-provided token (X_2), placing it in the second half of the residual stream:

$$\mathbf{W}_Q^{\text{user2}} = \begin{bmatrix} 0_{2 \times V} & \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{bmatrix}, \quad \mathbf{W}_K^{\text{user2}} = \begin{bmatrix} 0_{2 \times V} & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{bmatrix}, \quad \mathbf{W}_V^{\text{user2}} = \begin{bmatrix} 0_{V \times V} & 0_{V \times N} \\ 2\mathbf{I}_V & 0_{V \times N} \end{bmatrix},$$

where the factor 2 is there because, as usual, the first linear layer will subtract 1 from everything in order to extract the value selected by the first token.

This linear layer looks as usual: $\hat{\mathcal{L}}_{\text{ex2}} = \hat{\mathcal{L}}[\mathbf{I}_{2V}, -\mathbf{1}_{2V}]$. The result is that the first V elements will be 0 except one which designates which map from second user token to output we should use, and the second V elements have a one hot-encoding of the second user token. Constructing an MLP that unpacks the mapping can become quite involved so we do not provide an explicit form for it. But from the universal approximation theorems and the finiteness of the domain and range, we know that such an MLP should exist. We thus designate by `unpack` the MLP that decodes the first half of the residual stream to:

$$\left(\frac{m_{X_1}(1)}{V}, \dots, \frac{m_{X_1}(V)}{V} \right)$$

and keeps the second half unchanged.

And now, by using two attention heads, the second attention block extracts the value of the above vector at the position designated by the second token, in a fashion not dissimilar to all the previous cases:

$$\begin{aligned} \mathbf{W}_Q^{\text{emb}} &= [\mathbf{0}_V^\top, \mathbf{1}_V^\top], & \mathbf{W}_K^{\text{emb}} &= [\mathbf{1}_V^\top, \mathbf{0}_V^\top], & \mathbf{W}_V^{\text{emb}} &= [\mathbf{I}_V & 0_{V \times V}], \\ \mathbf{W}_Q^{\text{user2}'} &= [\mathbf{0}_V^\top, \mathbf{1}_V^\top], & \mathbf{W}_K^{\text{user2}'} &= [\mathbf{0}_V^\top, \mathbf{1}_V^\top], & \mathbf{W}_V^{\text{user2}'} &= [0_{V \times V} & \mathbf{I}_V], \end{aligned}$$

And finally, with $\hat{\mathcal{L}}_{\text{ex}} = \hat{\mathcal{L}}[\mathbf{I}_V, -\mathbf{1}_V]$, $\hat{\mathcal{L}}_{\text{sum}} = \hat{\mathcal{L}}[\mathbf{1}_V^\top, 0]$, and the same projection had as before (Equation (10)), we get the target token. The final transformer is then: $\mathcal{A}[(\mathbf{W}_Q^{\text{fst}}, \mathbf{W}_Q^{\text{user1}}, \mathbf{W}_Q^{\text{user2}}), (\mathbf{W}_K^{\text{fst}}, \mathbf{W}_K^{\text{user1}}, \mathbf{W}_K^{\text{user2}}), (\mathbf{W}_V^{\text{fst}}, \mathbf{W}_V^{\text{user1}}, \mathbf{W}_V^{\text{user2}})] \circ \hat{\mathcal{L}}_{\text{ex2}} \circ \text{unpack} \circ \mathcal{A}[(\mathbf{W}_Q^{\text{emb}}, \mathbf{W}_Q^{\text{user2}'}), (\mathbf{W}_K^{\text{emb}}, \mathbf{W}_K^{\text{user2}'}), (\mathbf{W}_V^{\text{emb}}, \mathbf{W}_V^{\text{user2}'})] \circ \hat{\mathcal{L}}_{\text{ex}} \circ \hat{\mathcal{L}}_{\text{sum}} \circ \mathcal{L}_{\text{proj}} \circ \text{softmax}$. You can see this transformer implemented and running in practice in Section 5 [here](#).